Numerical Linear Algebra Software

Present and Future

PIs: **Michael W. Mahoney** · **Riley J. Murray** · **Kimon Fountoulakis**

RandBLAS & RandLAPACK

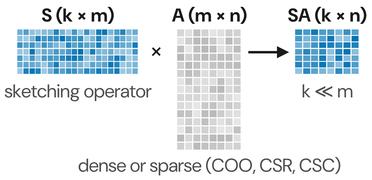
Building standard libraries for randomized NLA

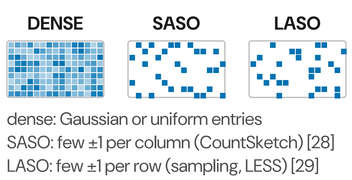
RandNLA (*Randomized Numerical Linear Algebra*) [3] uses randomness as a computational resource for faster, more robust matrix computations. It lays the foundation for the next generation of NLA; our libraries are a software instantiation of that foundation.

Responsible recklessness

Randomness as a tool buys speed and reliability, not uncertainty.

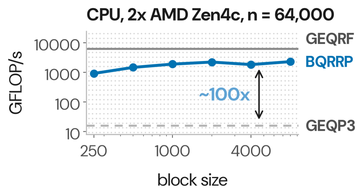
- **Sketching:** apply a random dimension-reducing map S to A. The sketch SA is far smaller yet keeps A's geometry (norms, angles, leading subspaces) with high probability, so the expensive step runs on a smaller problem.
- **RandBLAS 1.1.0** implements the operators and their products behind a BLAS-style API and aims to become a standard like the BLAS that vendors implement. Counter-based RNG (Random123 [8]: Philox by default, Threefry supported) makes draws reproducible.
- **Stability too:** our Q-less QR implementation [9] makes one Cholesky QR pass safe for any numerically full-rank operator: weaker assumptions than Cholesky QR2 [27], and one pass where sCholQR3 [23] needs three.
- **Where we are heading on GPUs:** kernels that generate and apply a sketch in one pass, and sketching fused with the operations that consume it.

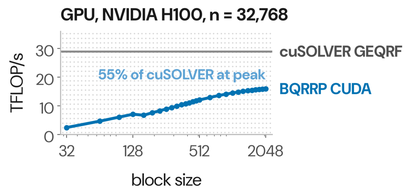




Outperforming standard methods

RandLAPACK implements, tests and benchmarks RandNLA methods, novel and known, that outperform their counterparts in standard NLA libraries.





Flagship showcase: BQRRP, randomized QR with column pivoting [1], vs vendor QR (GEQRF unpivoted, GEQP3 pivoted) on a dual AMD Zen4c CPU (left, log scale) and an NVIDIA H100 (right).

Task	RandLAPACK methods	Notes
Low-rank approximation	RSVD [20], ABRIK (block Krylov)	ABRIK matrix-free
Symmetric eigen, PSD	REVD2 [37], RPCholesky [21], Nyström EVD [3]	REVD2 matrix-free
QR with pivoting	BQRRP [1], CQRRPT [2], HQRRP [22]	BQRRP, CQRRPT on GPU
Tall and Q-less QR	CQRRT, CQRRTO [9], CholQR, sCholQR3 [23]	CQRRTO matrix-free
Least squares, regression	KRILL [24], Blendenpik [25], LSQR [26]	kernel callbacks
Matrix functions, traces	funNyström++ [7] (tr f(A), log-det), Lanczos	matrix-free
Building blocks	QB, range finders [20], preconditioners, PCG	

Interacting with the libraries

- **C++ at the core;** bindings for MATLAB (MEX) and Python, Julia planned. **GPUs:** RandLAPACK runs BQRRP and CQRRPT on NVIDIA CUDA; RandBLAS CUDA support is in development.
- **Vendor BLAS and LAPACK** through BLAS++ and LAPACK++ [34]: tested with OpenBLAS [38], Intel MKL [39], Apple Accelerate [40] and AMD AOCL [41] (BLIS [35] and libflame [36]); the wrappers also support Cray LibSci [42] and IBM ESSL [43].
- **One-command installers** on Linux, macOS and Windows; 330+ CI tests on all three; RandLAPACK's 1.0 release candidate targets November 2026.

100x
or more over LAPACK's pivoted QR (BQRRP on CPU) [1]

55%
of cuSOLVER's unpivoted QR speed on an H100 in our sweep (BQRRP) [1]

10/10
top kernels of a 2026 GPU QR competition wrong on legal inputs

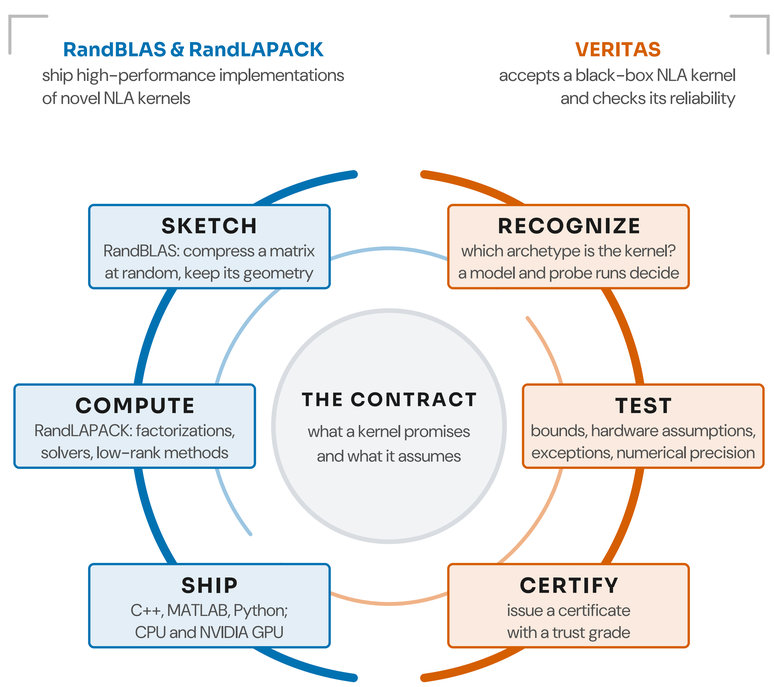
1,050
legal inputs tried per kernel, against the 19 the competition scored

10x
fewer matrix-vector products than a standard trace estimator at condition number 10^7 (funNyström++) [7]

1700x
faster than scikit-learn [32] kernel PCA at 100,000 points (RPCholesky [21]; 3 GB of RAM at 240,000)

40k
theorems and lemmas in the VSCL Lean library [6]; Mathlib has 13

13
theorem lines on numerical analysis in all of Mathlib [19] (182,000 total)



For fifty years numerical linear algebra meant expert-written, ***standardized libraries***, trusted through test suites and stability proofs. Generative AI now makes a kernel ***nearly free to produce and expensive to trust***: it passes its developer's tests and ***fails silently*** on hard inputs, in reduced precision, or on other hardware. So we continue building the ***next generation of NLA libraries***, and develop the ***means to trust any kernel***.

Project links

- one-command install of both libraries
- MATLAB and Python bindings
- the BLAS grading paper [5] and test code
- the VSCL Lean library
- **sources [1] to [45]**

VERITAS

Bringing trust to NLA kernels

VERITAS (*Verification Engine for Reliable and Intelligent Testing of Algebra Systems*) is a **proposed** framework for testing and certifying numerical kernels, whoever or whatever wrote them.

The motivation

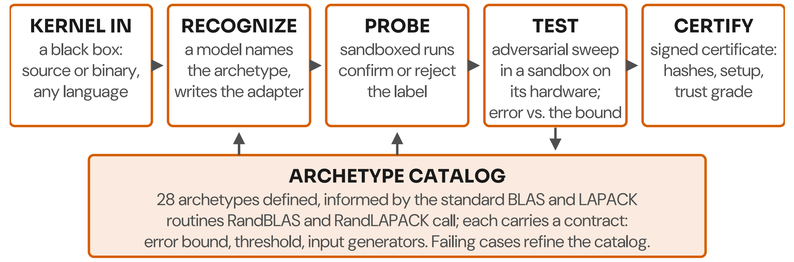
Round 2 of a 2026 GPU fast-QR competition [4]: top-5 kernels' speedup over cuSOLVER and wrong factorizations on about 1,050 legal inputs the scored set never covered

Leaderboard	Scored	Speedup	# Failed tests
rank 1	19/19	107x	157
rank 2	19/19	106x	46
rank 3	19/19	102x	330
rank 4	19/19	91x	3
rank 5	19/19	87x	360

rank 4 fails only 3 times: it detects hard inputs and falls back to an exact routine

- All five pass every scored case; on about 1,050 legal inputs from the organizers' generator (checker unchanged) all five fail.
- Cholesky factorization [44] and eigendecomposition [45] follow-ups: 7 of 10 broken; two of the rest hold only by falling back to an exact method on hard inputs, the last fails only in long input sequences. We aspire to automate such audits, ending in a signed certificate.

The proposed design



- **Archetypes:** BLAS 1 to 3, LAPACK factorizations and solvers (LU, Cholesky, QR, eigen, SVD), sparse kernels (SpMV, SpMM, SpGEMM, solves), sketching.
- **Grading:** the A, B, C grades of the BLAS [5] are our reference. We intend to extend them to more assumptions (precision, exceptions), more hardware and every archetype. Closed kernels expose no internals, so the grade is the only model of them, and the tests must resist gaming: secret seeds, mixed inputs, adversarial search.

VSCL: the theorems behind the tests

The Verified Scientific Computing Library (VSCL) [6] puts numerical analysis in Lean 4 [33]; its formalized results are the basis of VERITAS tests.

1.6M
nonblank lines of Lean 4 on top of Mathlib

40,478
theorems and lemmas; 0 sorry, 0 axioms

- **Formalized:** Higham [10] (28 chapters), LeVeque [11], Vershynin [12], Drineas and Mahoney [13]. **In progress:** Nocedal and Wright [14], Bhatia [15]. **Next:** Greenbaum [16], Saad [17], Boyd [18].
- **Proved today:** backward-error bounds for summation, triangular solves, Householder QR and Cholesky; error bounds for Drineas-Mahoney sampling.
- **How it is built:** a book-formalization skill drives coding agents chapter by chapter (extract, inventory, prove in Lean 4, audit faithfulness, publish); open-weight provers are under test.
- **Benchmark:** on 10 stability-proof tasks, an agent with an early version of the library proved 6.1x faster with 5.4x shorter proofs.
- **Next targets:** error bounds for IEEE P3109 [31] low-precision reductions; LU and QR on BLAS of a known grade; the MR³ SVD [30]; mixed and very low precision.